

Ejercicio #3

Debugging y Performance

Revisa el siguiente código (teórico) y encuentra 3 errores de lógica/performance

```
@Component {
```

```
  template: '
```

```
    <div *ngfor = "let item of items">
```

```
      {{expensiveCalculation(item)}}
    </div>
```

```
  ,
```

```
  ,
```

```
} )
```

```
export class ListComponent {
```

```
  @input() items: any[];
```

DICIEMBRE DECEMBER

V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D
F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

```
expensiveCalculation(item) {  
  console.log("Calculando..."); //simula proceso  
  return Math.random();         pesado.  
}
```

```
}
```

SOLUCIÓN

- 1- Error Crítico de Performance: Función en el template: Llamar la función `expensiveCalculation(item)` directamente en la interpolación del HTML es una pésima práctica en Angular.
- ¿Porque ocurre?: Angular ejecuta el Change Detection ante cualquier evento Asíncrono. Al estar la función en el template, Angular volverá a ejecutar `expensiveCalculation(item)` una y otra vez

Un actor es un señor que hoy
come faisán y mañana se come las plumas.

Fidel Pintos

06 Solución ideal: Calcular estos valores
 08 previamente en el typescript (transformando
 09 los datos al recibirlos en el @input) o la mejor
 10 opción, crear un pipe puro (@pipe({pure: true}))
 para realizar el cálculo, ya que los pipes puros
 memorizan el resultado y solo se recalculan
 01 si la referencia del item cambia.

02

03 2-El error lógico Oculto:

04 ExpressionChangedAfterItHasBeenCheckedError

05 Problema: La función está retornando,
 06 Mat.random().

07

DICIEMBRE DECEMBER

V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D
F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

¿Por qué Ocurre?: En modo desarrollador, Angular realiza un ciclo de verificación doble para asegurar que los datos fluyan en una sola dirección y no haya efectos secundarios inesperados durante el renderizado. Como `Math.random()` devuelve un número distinto cada vez que se llama, el valor del primer chequeo será diferente al del segundo chequeo. Esto lanzará un error fatal en consola y romperá la aplicación.

3- Fuga de Rendimiento en el renderizado:
(falta de trackBy)

El problema: Uso de la directiva `<div *ngfor="let item of items">` sin una función de rastreo.

¿Por qué Ocurre?: Si el arreglo `items` muta, Angular por defecto no sabe qué elemento específico cambió. En consecuencia, destruirá todos los elementos del DOM de esa lista y los volverá a crear desde 0.

Solución ideal: Crear una función `trackBy` (`trackBy | itemId`).

DICIEMBRE DECEMBER

V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D							
F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

*

16
SÁBADO
SATURDAY

350 / 15

DECEMBER



07 Bonus

08 Uso de `@input()` items: any[];
09 destruye el tipado estricto de typescript.
10
11
12

17
DOMINGO
SUNDAY

351 / 14

DICIEMBRE
DECEMBER

El éxito es fácil de obtener.
Lo difícil es merecerlo.
Albert Camus